PRÁCTICA 5

Segmentación de cauce en procesadores RISC. CPI. Atascos RAW y forwarding

Objetivos *Comprender el funcionamiento de la segmentación de cauce del procesador MIPS de 64 bits. Analizar las ventajas e inconvenientes de este tipo de arquitectura.*

CPI

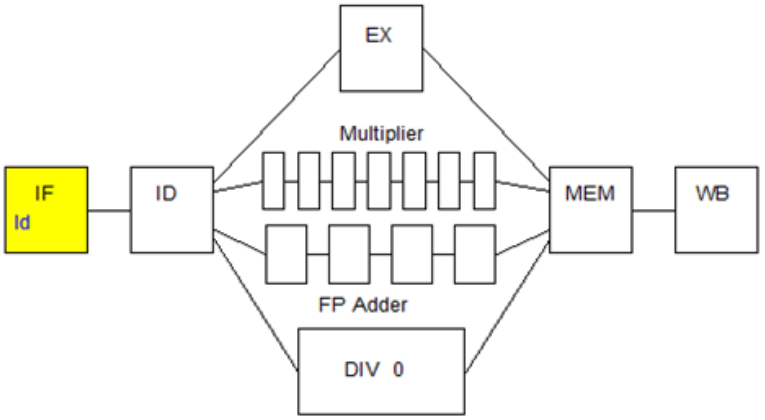
Los Ciclos Por Instrucción (CPI) son una medida de uso eficiente del procesador. Se definen como

$$\text{CPI} = \frac{Ciclos}{Instrucciones}$$

De esta forma, un cauce de un procesador es más eficiente que otro si su CPI es más *bajo, es decir,* requiere menos ciclos para las mismas instrucciones.

Valor mínimo del CPI En arquitecturas tradicionales, el CPI no puede ser menor que 1, ya que en el mejor caso se termina una instrucción distinta en cada ciclo. En arquitecturas con múltiples unidades de ejecución si puede ser menor a 1.

La arquitectura del simulador WinMIPS tiene un pipeline de 5 etapas para las instrucciones más simples que usan la etapa EX, donde cada etapa requiere 1 ciclo para completarse si no hay atascos..



Por ese motivo, generalmente para un programa sin atascos el CPI puede calcularse como:

$$\text{CPI} = \frac{Ciclos}{Instrucciones} = \frac{Instrucciones + 4}{Instrucciones}$$

En este caso, el valor 4 proviene de que en los primeros 4 ciclos no se termina ninguna instrucción

Valor máximo del CPI En teoría, el CPI no tiene un límite superior, y podría ser tan grande como la ineficiencia del procesador. Idealmente, para un programa optimizado el CPI debería ser cercano a 1. Por ejemplo, un valor común para un programa optimizado sería 1.1 o 1.2. No obstante, esos valores de CPI generalmente solo se obtienen si un programa requiere ejecutar una gran cantidad de instrucciones que no se **atasquen** demasiado.

Atascos Las instrucciones pueden atascarse por diversos motivos, principalmente por falta de datos requeridos para ejecutarse. Cuando una instrucción se **atasca**, la misma no puede avanzar y por ende requiere más ciclos para ejecutarse, aumentando el CPI. En una arquitectura donde las etapas de las instrucciones duran 1 ciclo, entonces el número de ciclos requeridos para ejecutar la instrucción aumenta en 1 por cada atasco. En ese caso, para calcular el CPI podemos asumir:

$$\text{CPI} = \frac{Ciclos}{Instrucciones} = \frac{Instrucciones + 4}{Instrucciones} = \frac{Instrucciones + Atascos + 4}{Instrucciones}$$

Esta fórmula solo funciona cuando las instrucciones pasan todas por las mismas 5 unidades de ejecución básicas del WinMIPS64; en particular, NO VA a funcionar así cuando existen multiplicaciones o divisiones. A continuación, veremos los distintos tipos de atascos que pueden darse en la ejecución de los programas y cómo minimizarlos.

Parte 1: Estructura del Cauce

Para comprender cómo se puede atascar una instrucción, primero se debe comprender qué parte de la instrucción se ejecuta en cada etapa del cauce.

1. Funciones de las etapas del cauce⭐

Indicar a qué etapa corresponde cada función que realiza el procesador al ejecutar una instrucción típica:

Función	Etapa
A. Decidir si se toma un salto o no B. Buscar la instrucción en memoria y llevarla a la CPU C. Calcular la dirección de un acceso a memoria D. Guardar el valor de un registro en memoria E. Traer de memoria un valor a un registro intermedio F. Almacenar el valor final de un registro G. Calcular la dirección de un salto H. Verificar si están disponibles los operandos necesarios para continuar con la ejecución de la instrucción	1. IF: Obtención de Instrucción 2. ID: Decodificación de Instrucción 3. EX: Ejecución principal de instrucción 4. MEM: Acceso a Memoria 5. WB: Escritura de resultado en registro destino

2. Etapas de una instrucción ⭐

Indicar, para las siguientes instrucciones, qué realizan en cada una de las etapas IF/ID/EX/MEM/WB.

Instrucción	IF	ID	EX	MEM	WB
`daddi $t1, $t3, 4`					
`dadd $t2, $t4, $t3`					
`sd $t3, tabla($t2)`					
`ld $t3, tabla($t2)`					
`bneq $t1, $t2, loop`					
`j loop`					

3. Cálculo de CPI ⭐

Simular la ejecución del siguiente programa **manualmente** (sin usar el simulador), dibujando el cauce como lo hace el simulador, y calculando la cantidad de instrucciones, ciclos, y CPIs.

```
daddi $t2,$t3,5
dsub $t4,$t3,$t5
xor $t6,$t3,$t5
nop
halt
```

Responder:

- La fórmula de CPI presentada anteriormente ¿se cumple para este programa?
- La instrucción HALT solo detiene la ejecución del programa ¿se cuenta en el cálculo del CPI?
- El simulador considera a la instrucción NOP para calcular el CPI. No obstante, dicha instrucción no realiza ninguna tarea. Agregar 20 instrucciones NOP más, y calcular nuevamente el CPI. ¿Qué valor toma? En el caso de que sea menor, ¿eso quiere decir que el programa es más eficiente?

3. Cálculo de CPI con divisiones y multiplicaciones⭐

Simular la ejecución del siguiente programa, y observe la cantidad de instrucciones, ciclos, y CPIs.

```
ddiv $t6,$t3,$t5
dmul $t4,$t3,$t5
daddi $t2,$t3,5
halt
```

Responder:

- La fórmula de CPI presentada anteriormente ¿Se cumple para este programa?
- Se utilizan las instrucciones ddiv y dmul ¿Cuántos se requieren para su ejecución?
- ¿Por qué la instrucción dmul tiene varias etapas llamadas M1, M2, etc, y ddiv no?
- Modificar el programa, duplicando las instrucciones ddiv y dmul, de manera que cada una aparezca dos veces, y volver a simular. ¿Qué diferencias hay entre la multiplicación y la división??

Parte 2: Atascos RAW

1. Dependencia de datos ⭐

Los **atascos RAW** (*read after write*) son los más comunes en un programa. Para determinar si hay un atasco RAW entre dos instrucciones, primero hay que determinar si hay una **dependencia de datos de lectura** entre las mismas.

Las dependencia de datos de lectura ocurren cuando una instrucción B requiere el valor de un registro, pero debe esperar a que otra instrucción A escriba el valor de ese registro. En este caso, B es una instrucción que comienza a ejecutarse luego de A.

La dependencia de datos no siempre implica un atasco, porque si las instrucciones están muy alejadas en el tiempo la primera termina antes de que la segunda necesite el dato.

a) Los siguientes programas cortos contienen instrucciones que utilizan registros similares. Indicar en cada caso qué instrucciones tienen dependencias de datos de lectura entre ellas.

	Programa A		**Programa B**
1	`daddi $t1, $0, 5`	1	`ld   $t1, A($0)`
2	`daddi $t2, $0, 7`	2	`dsll $t1, $t1, 2`
3	`slt  $t3, $t1, $t2`	3	`ld   $t2, B($0)`
4	`daddi $t1, $0, 1`	4	`dsub $t1, $t1, $t2`
5	`and  $t4, $t3, $t1`	5	`ld   $t3, C($0)`
6	`daddi $t1, $t0, 8`	6	`dadd $t1, $t1, $t3`
7	`sd   $t4, A($t1)`	7	`sd   $t1, R($0)`

Por ejemplo, para el programa A:

3 depende de 1 (por $t1) y 2 (por $t2)
5 depende de 3 (por $t3) y 4 (por $t1)
7 depende de 5 (por $t4) y 6 (por $t1)

b) ¿Cuáles de las dependencias de datos por lectura te parece que causarán atascos RAW? Probar los programas en el simulador y anotar la cantidad de atascos y CPI de cada uno.

c) Modificar los programas para que se reduzca la cantidad de atascos RAW, reordenando las instrucciones de forma que el resultado final del programa sea el mismo. Comparar la cantidad de atascos y CPI de cada uno con el caso anterior.

Parte 3: Atascos por Salto

Cuando la computadora se encuentra con un salto, se dice que el programa se bifurca en ramas y tiene 2 opciones: (1) puede seguir ejecutando el programa consecutivamente o (2) saltar y continuar la ejecución en otro punto. En WinMIPS64, la CPU puede determinar si el salto debe tomarse en la etapa **ID**, pero esto genera un problema importante: cuando la instrucción de salto pasa a la etapa ID, ¿qué instrucción se carga en la etapa IF? Por defecto, WinMIPS64 carga la instrucción que sigue en el orden del código.

Si se determina que no se debe saltar, el programa sigue sin inconvenientes. Pero si se debe saltar, se debe **descartar la instrucción en IF** y volver a cargar la instrucción a donde se salta. Este "ciclo perdido" de la instrucción que se descarta se contabiliza como un **BTS** (*branch taken stall*).

Nota: los saltos incondicionales también generan BTS. Esto se debe a que la CPU recién puede ejecutar el salto en ID.

1. BTS en un salto condicional tipo IF⭐

```
1         .data
2   A: .word 0
3   B: .word 0
4   .code
5         ld    $t1, A($0)
6         ld    $t2, B($0)
7
```

```
8           bne   $t1, $t2, no
9           daddi $t3, $0, 1
10          j     fin
11      no: daddi $t3, $0, 0
12      fin: sd   $t3, C($0)
13          halt
14
```

a) Analice el programa considerando A=1 y B=2. ¿Cuántos BTS ocurren? ¿Cuál es el CPI?

b) Repita a) con A=2 y B=1.

c) ¿Cómo modificaría el programa para reducir los BTS? Puede agregar, modificar o quitar cualquier línea.

2. BTS en un bucle *for* ⭐

```
1         .data
2  res: .word 0
3  .code
4         daddi $t1, $0,  4
5         daddi $t2, $0,  3
6         daddi $t3, $0,  0
7
```

```
8    loop: dadd  $t3, $t3, $t2
9          daddi $t1, $t1, -1
10         bnez  $t1, loop
11         sd    $t3, RES($0)
12         halt
13
14
```

a) ¿Cuántos BTS ocurren en el programa?

b) Viendo el código del programa de forma estática (es decir, sin ejecutarlo), la cantidad de instrucciones que tiene sería 8. No obstante, las estadísticas del simulador indican que hay más instrucciones, ¿cuántas? ¿De dónde sale esta diferencia? ¿Cómo calcularías la cantidad de instrucciones de un programa con un lazo?

c) ¿Qué instrucciones generan los atascos tipo RAW y por qué? ¿En qué etapa del cauce se produce el atasco en cada caso y durante cuántos ciclos?

d) Considerando lo visto en b) y c), ¿cuál es el CPI?

e) Modifique el operando inmediato de la línea 1 al valor 1000. ¿Cuántas instrucciones se indican en el simulador? ¿Cómo cambió el CPI?

3. BTS en un bucle *while* ⭐

```
1        .data
2  A: .word 1,3,2,0
3  res: .word
4  .code
5  daddi $t1, $0, 0
6  daddi $t2, $0, 0
7
8   loop: ld    $t3, A($t1)
9         beqz  $t3, fin
10        dadd  $t2, $t2, $t3
11        daddi $t1, $t1, 8
12        j     loop
13  fin:  sd    $t2, RES($0)
14        halt
```

a) ¿Qué realiza el programa? ¿Qué determina la cantidad de iteraciones del bucle?

b) ¿Cuántas instrucciones de salto hay en el programa? ¿Cuáles generan BTS?

c) ¿Qué instrucciones generan los atascos tipo RAW y por qué? ¿En qué etapa del cauce se produce el atasco en cada caso y durante cuántos ciclos?

c) Pruebe declarando distintos valores para A y observe cómo cambia el CPI. ¿Puede lograr que el CPI sea menor que 1,6?

Parte 4: Forwarding

Previamente, vimos que reordenando las instrucciones se puede obtener una mejora en el CPI del programa. Otra forma de solucionar los atascos por dependencia de datos es utilizando el Adelantamiento de Operandos o *Forwarding*. Un procesador con Forwarding tiene un hardware modificado que permite que menos dependencias de datos se conviertan en atascos RAW. Para lograrlo, utiliza dos estrategias complementarias:

1) El valor que se calcula en las etapas EX o MEM está disponible para que otras instrucciones lo accedan ni bien se calcula, y no se requiere esperar a la etapa WB. Por ejemplo, la instrucción **daddi $t1,$t2,5** tendrá disponible el resultado de la suma al finalizar la etapa **EX**.

2) Las instrucciones no requieren todos sus operandos en la etapa **ID**. En lugar de eso, si necesitan un operando y no está disponible, se atascarán en la etapa en que lo necesiten realmente. Por ejemplo, si la instrucción **sd $t1, A($0)** llega a la etapa **ID** pero el valor de **$t1** todavía está siendo calculado por otra instrucción, avanzará igual y se atascará recién en la etapa **MEM.**

1. Forwarding básico ⭐

Para analizar el efecto del forwarding, veamos el siguiente programa que intercambia el contenido de dos palabras de la memoria de datos, etiquetadas A y B.

```
    .data
A:  .word  1
B:  .word  2

    .code
    ld   $t1, A($0)
    ld   $t2, B($0)
    sd   $t2, A($0)
    sd   $t1, B($0)
    halt
```

a) Ejecutarlo en el simulador con la opción Configure/Enable Forwarding **inhabilitada**.

- ¿Cuántos atascos RAW hay?
- ¿Cuál es el CPI?

b) Ejecutarlo en el simulador con la opción Configure/Enable Forwarding **habilitada**.

- ¿Por qué no se presenta ningún atasco en este caso? Explicar la mejora.
- ¿Cuál es el CPI?
- ¿Qué indica el color de los registros en la ventana Register durante la ejecución?

2. Forwarding con lazos ⭐⭐

Analizar el siguiente programa:

```
    .data
A:  .word 1
B:  .word 3
C:  .word 0
```

```
      .code
      ld    $t1, A($0)
      ld    $t2, B($0)
loop: dsll  $t1, $t1, 1
      daddi $t2, $t2, -1
      bnez  $t2, loop
      sd    $t1, C($0)
      halt
```

a) **Loop sin forwarding** Ejecutar el programa deshabilitando el Forwarding y responder:

- ¿Qué instrucciones generan los atascos tipo RAW y por qué? ¿En qué etapa del cauce se produce el atasco en cada caso y durante cuántos ciclos?
- ¿Cuántos CPI tiene la ejecución del programa en este caso?
- Cambiar el valor de B a 1000 y ejecutar. ¿Cómo cambió la cantidad de CPIs?

b) **Loop con forwarding** Ejecutar el programa con Forwarding habilitado y responder:

- ¿Cuántos CPI tiene la ejecución de este programa? Tomar nota del número de ciclos, cantidad de instrucciones y CPI. Comparar con el caso anterior.
- Cambiar el valor de B a 1000 y ejecutar. ¿Cómo cambió la cantidad de CPIs?

c) **Reordenamiento de instrucciones para optimización de CPI:**

- Reordenar las instrucciones para que la cantidad de RAW sea 0 en la ejecución del programa, ejecutando con Forwarding **habilitado** y B=3.
- Cambiar el valor de B a 1000 y ejecutar. ¿Cómo cambió la cantidad de CPIs?
- En base a lo anterior ¿qué partes del programa conviene optimizar generalmente?

3. Optimización de CPI ⭐⭐⭐

El siguiente programa copia un arreglo de **src** a **dest**.

```
1         .data
2   src:  .word   1, 2, 3, 4, 5, 6, 7, 8
3   dest: .space  64
4
5         .text
6         daddi $t0, $0, 12
7         daddi $t3, $0, src
8         daddi $t2, $0, dest
9   loop: ld    $t1, 0($t3)
10        sd    $t1, 0($t2)
11        daddi $t3, $t3, 8
12        daddi $t2, $t2, 8
13        daddi $t0, $t0, -1
14        bnez  $t0, loop
15        halt
```

a) Ejecute el programa con **forwarding habilitado**. ¿Cuántos atascos ve? ¿Cuál es el CPI?

b) Únicamente reordenando instrucciones, ¿puede lograr eliminar todos los atascos RAW?